

Go Back

Hardware

Nicla Vision

Tutorials

Blob Detection with OpenMV

Smart Elevator Monitoring System with the Portenta Proto Kit

Getting Started with Nicla Vision

Image Classification with Edge Impulse®

Debugging with Lauterbach TRACE32 GDB Front-End Debugger

Video Live Streaming with OpenMV and Nicla Vision

Reading Audio Samples With the Onboard Microphone

Accessing IMU Data on Nicla Vision

Proximity Detection with Arduino Nicla Vision

Voice Commands With The Arduino Speech Recognition Engine

Nicla Vision User Manual

Home / Hardware / Nicla Vision / Nicla Vision User Manual

Nicla Vision User Manual

Learn about the hardware and software features of the Arduino® Nicla Vision.

Author · Christopher Méndez · Last revision · 10/03/2024

Overview

This user manual will guide you through a practical journey covering the most interesting features of the Arduino Nicla Vision. With this user manual, you will learn how to set up, configure and use this Arduino board.

Hardware and Software Requirements

Hardware Requirements

- Arduino Nicla Vision (x1)
- Micro USB cable (x1)

Software Requirements

- OpenMV IDE
- Arduino IDE 1.8.10+, Arduino IDE 2.0+, or Arduino Cloud Editor

To create custom Machine Learning models, the Machine Learning Tools add-on integrated into the Arduino Cloud is needed. In case you do not have an Arduino Cloud account, you will need to create one first.

ON THIS PAGE

Overview

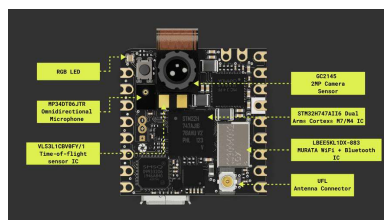
- Hardware and Software Requirements +
- Product Overview +
- First Use +
- Pins +
- Onboard Sensors +
- Machine Learning Tools +
- Communication +
- Arduino Cloud +
- Support +

The Nicla Vision is a ready-to-use, standalone camera board for analyzing and processing images on the edge. Thanks to its 2 MP color camera, smart 6-axis motion sensor, integrated microphone, and distance sensor, it is suitable for asset tracking, object recognition, and predictive maintenance.

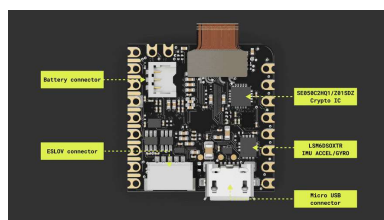
The Nicla Vision lets you quickly implement sensor nodes to send collected data to the Arduino® Cloud (or third-party vendor services) via its onboard Wi-Fi® and Bluetooth® module.

Board Architecture Overview

The Nicla Vision features a robust and efficient architecture that integrates a range of sensors packed into a tiny footprint. Nicla Vision combines a powerful STM32H747AI16 Dual Arm® Cortex®-M7/M4 IC processor with a 2MP color camera that supports TinyML, as well as a smart 6-axis motion sensor, integrated PDM microphone and a Time of Flight distance sensor.



Nicla Vision main components (top view)



Nicla Vision main components (bottom view)

Here is an overview of main components of the board, as shown in the images above:

Camera: the Nicla Vision features a camera based on the GC2145 Color

and an image signal processor. The 2MP GC2145 CMOS camera module is equipped with an 80° (DFOV) stock lens, 1.75 μm pixel size and a focal length of 2.2 mm. It supports RGB output format.

Microcontroller: the heart of the Nicla Vision is the dual-core STM32H747 (U1), including an Arm® Cortex®-M7 running at 480 MHz and an Arm® Cortex®-M4 running at 240 MHz. The two cores communicate via a Remote Procedure Call mechanism that allows calling functions on the other processor seamlessly.

Onboard advanced motion sensor: the board features the LSM6DSOX, a smart IMU that includes a 3-axis accelerometer and a 3-axis gyroscope. The LSM6DSOX has a full-scale acceleration range of $\pm 2/\pm 4/\pm 8/\pm 16$ g and an angular rate range of $\pm 125/\pm 250/\pm 500/\pm 1000/\pm 2000$ dps.

Onboard distance sensor: the VL53L1CBV0FY Time-of-Flight sensor (U4) adds accurate and low-power ranging capabilities to the Nicla Vision. The invisible near infrared VCSEL laser (including the analog driver) is encapsulated together with receiving optics in an all-in-one small module located below the camera.

Digital Microphone: the MP34DT05 digital MEMS microphone (U6) is omnidirectional and operates via a capacitive sensing element with a high (64 dB) signal-to-noise ratio. The sensing element, capable of detecting acoustic waves, is manufactured using a specialized silicon micromachining process dedicated to audio sensors production.

Wireless connectivity: the Murata® LBEE5KL1DX-883 wireless module (U9) simultaneously provides Wi-Fi® and Bluetooth® connectivity in an ultra-small package based on the Cypress CYW4343W. The IEEE802.11 b/g/n Wi-Fi® interface can operate as an access point (AP), station (STA), or dual-mode simultaneous AP/STA. It supports a maximum transfer rate of 65 Mbps. Bluetooth® interface supports Bluetooth® Classic and Bluetooth® Low Energy. An integrated antenna circuitry switch allows a single external antenna (J6) to be shared between Wi-Fi® and Bluetooth®.

Power management: the Nicla Vision is

features that ensure minimal energy consumption even when using always-on motion recognition and image processing. The Nicla Vision features the PF1550 from NXP®, a highly integrated battery charge management integrated circuit (IC) designed for wearables and Internet of Things (IoT) devices.

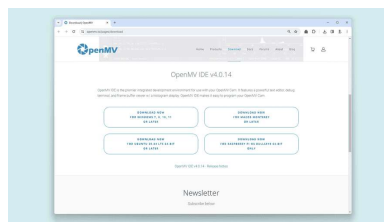
Security Elements: the Nicla Vision enables IC level edge-to-cloud security capability through the NXP SE050C2 Crypto chip (U8). This provides Common Criteria EAL 6+ security certification up to OS level, as well as RSA/ECC cryptographic algorithm support and credential storage.

Board Core and Libraries

With OpenMV IDE

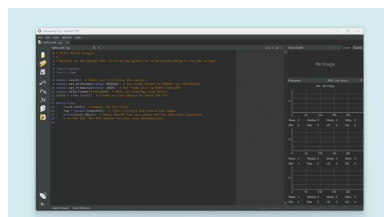
Before you can start programming MicroPython scripts for the Nicla Vision, you need to download and install the OpenMV IDE.

Open the [OpenMV](#) download page in your browser, download the latest version available for your operating system, and follow the instructions of the installer.



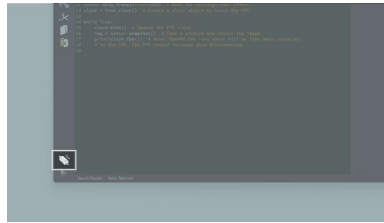
OpenMV Download Page

Open the OpenMV IDE and connect the Nicla Vision to your computer via the USB cable if you have not done so yet.



The OpenMV IDE after starting it

Click on the "connect" symbol at the bottom of the left toolbar.



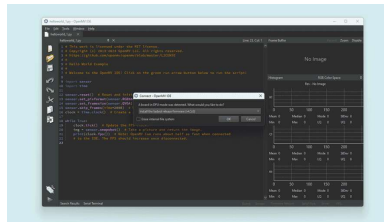
Click the connect button to attach the Nicla Vision to the OpenMV IDE

If your Nicla Vision does not have the latest firmware, a pop-up will ask you to install it. Your board will enter in DFU mode and its green LED will start fading.

Select

Install the latest [release](#) firmware.

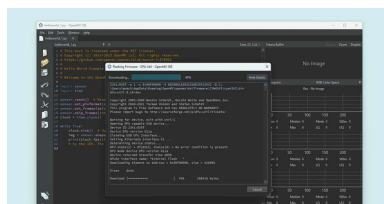
This will install the latest OpenMV firmware on the Nicla Vision. You can leave the option of erasing the internal file system unselected and click **OK**.



Install the latest version of the OpenMV firmware

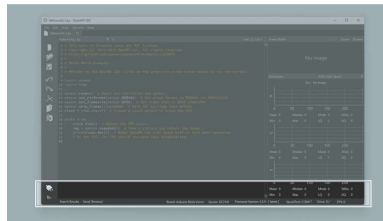
Nicla Vision's green LED will start flashing while the OpenMV firmware is being uploaded to the board. A loading bar will start showing you the flashing progress.

Wait until the green LED stops flashing and fading. You will see a message saying **DFU firmware update complete!** when the process is done.



Installing firmware on Nicla Vision board in OpenMV

The board will start flashing its blue LED when it is ready to be connected. After confirming the completion dialog, the Nicla Vision should already be connected to the OpenMV IDE, otherwise, click the "connect" button (plug symbol) once again (the blue blinking should stop).



When the Nicla Vision is successfully connected a green play button appears

While using the Nicla Vision with OpenMV, the RGB LED of the board can be used to inform the user about its current status. Some of the most important ones are the following:

● **Blinking Green:** Your Nicla Vision onboard bootloader is running. The onboard bootloader runs for a few seconds when your Nicla Vision is powered via USB to allow OpenMV IDE to reprogram your Nicla Vision.

● **Blinking Blue:** Your Nicla Vision is running the default **main.py** script onboard.

If you overwrite the **main.py** script on your Nicla Vision, then it will run whatever code you loaded on it instead.

If the LED is blinking blue but OpenMV IDE cannot connect to your Nicla Vision,

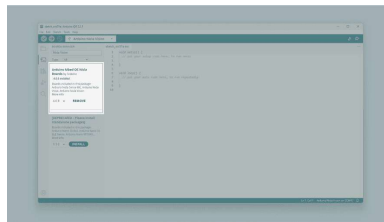
i please make sure you are connecting your Nicla Vision to your PC with a USB cable that supplies both data and power.

● **Blinking White:** Your Nicla Vision firmware is panicking because of a hardware failure. Please check that your Nicla Vision's camera module is installed securely.

If you tap the Nicla Vision reset button once, the board resets. If you tap it twice,  the board enters in Device Firmware Upgrade (DFU) mode and its green LED starts blinking and fading.

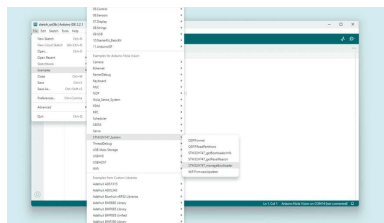
With Arduino IDE

The **Arduino Mbed OS Nicla Boards** core contains the libraries and examples you need to work with the board's components, such as its camera and IMU. To install the core for Nicla boards, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `Nicla Vision` and install the latest `Arduino Mbed OS Nicla Boards` version.



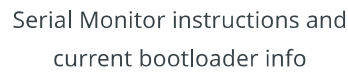
Installing the Arduino Mbed OS Nicla Boards core in the Arduino IDE

To update the bootloader firmware of your Nicla Vision, go to **File > Examples > STM32H747_System > STM32H747_manageBootloader** and upload this sketch to your board.



Example sketch location for bootloader update

After the sketch is uploaded, follow the instructions in the Serial Monitor.

[illegible]

Nicla Vision pinout

Nicla Vision full pinout

Nicla Vision datasheet

Nicla Vision schematics

The complete STEP files are available and downloadable from the link below:

First Use

Powering the Board

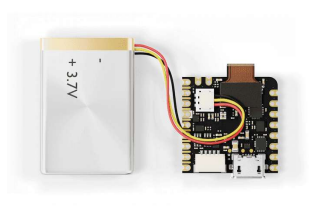
The Nicla Vision can be powered by:

Using a Micro USB cable (not included).

Using an external **5 V power supply** connected to `VIN` pin (please, refer to the [board pinout section](#) of the user manual).

Using a **3.7 V Lithium Polymer (Li-Po) battery** connected to the board through the onboard battery connector; the manufacturer part number of the battery connector is BM03B-ACHSS and its matching receptacle manufacturer part number is ACHR-03V-S. The **recommended minimum battery capacity for the Nicla Vision is 200 mAh**. A Li-Po battery with an integrated NTC thermistor is also recommended for thermal protection.

Using the onboard **ESLOV connector**, which has a dedicated 5V power line.



Nicla Vision battery powered

Hello World Example

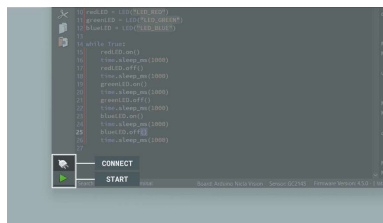
Let's program the Nicla Vision with the classic `hello world` example used in the Arduino ecosystem: the `Blink` sketch. We will use this example to verify the board's connection to the IDEs and that the Nicla Vision core and the board itself are working as expected.

With OpenMV

Copy and paste the code below into a new sketch in the OpenMV IDE.

```
1 import time
2 from machine import LED
3
4 redLED = LED("LED_RED")
5 greenLED = LED("LED_GREEN")
6 blueLED = LED("LED_BLUE")
7
8 while True:
9     redLED.on()
10    time.sleep_ms(1000)
11    redLED.off()
12    time.sleep_ms(1000)
13    greenLED.on()
14    time.sleep_ms(1000)
15    greenLED.off()
16    time.sleep_ms(1000)
17    blueLED.on()
18    time.sleep_ms(1000)
19    blueLED.off()
20    time.sleep_ms(1000)
```

To run the code on the Nicla Vision, click the **Connect** button and then the **Start** button.



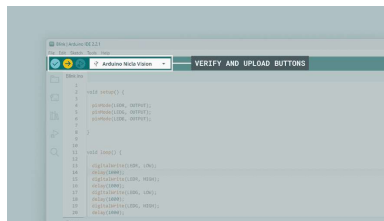
Running the Python Script on
OpenMV

With Arduino IDE

Copy and paste the code below into a new sketch in the Arduino IDE.

```
1 void setup() {  
2  
3   pinMode(LED_R, OUTPUT);  
4   pinMode(LED_G, OUTPUT);  
5   pinMode(LED_B, OUTPUT);  
6  
7   digitalWrite(LED_R, HIGH);  
8   digitalWrite(LED_G, HIGH);  
9   digitalWrite(LED_B, HIGH);  
10  
11 }  
12  
13 void loop() {  
14  
15   digitalWrite(LED_R, LOW); //  
16   delay(1000);  
17   digitalWrite(LED_R, HIGH);  
18   delay(1000);  
19   digitalWrite(LED_G, LOW);  
20   delay(1000);  
21   digitalWrite(LED_G, HIGH);  
22   delay(1000);  
23   digitalWrite(LED_B, LOW);  
24   delay(1000);  
25   digitalWrite(LED_B, HIGH);  
26   delay(1000);  
27  
28 }
```

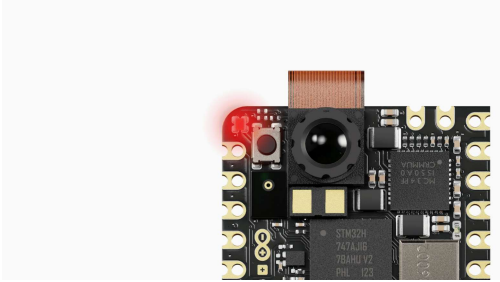
To upload the code to the Nicla Vision, click the **Verify** button to compile the sketch and check for errors; then click the **Upload** button to program the board with the sketch.



Uploading a sketch to the Nicla Vision in the Arduino IDE

Results

You should now repeatedly see the onboard LED turning red, green, and blue.



Pins

Analog Pins

The Nicla Vision has **three analog input pins**, mapped as follows:

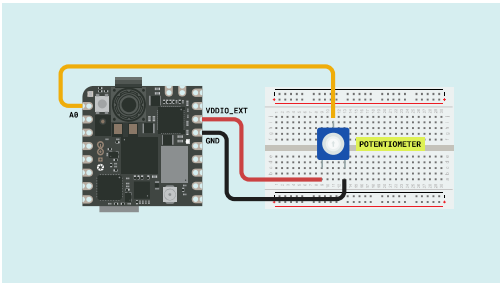
Microcontroller Pin	Arduino Pin Mapping
ADC1/PC_4	A0
ADC2/PF_13	A1
ADC3/PF_3	A2

All of them can be used through the built-in functions of the Arduino programming language.

i

The Nicla Vision ADC reference voltage is fixed to 3.3V, this means that it will map the ADC range from 0 to 3.3 volts.

We will use the Nicla Vision analog inputs on both IDEs, OpenMV and Arduino. For the example codes shown below, we will be reading the analog input `A0` and displaying the read voltage on the Serial Monitor of both IDEs:



With OpenMV

(it's maximum).

This example code can also be found on **File**
> **Examples** > **Board Control** >
adc_read_ext_channel.py:

```
1 import time
2 from pyb import ADC
3
4 adc = ADC("A0") # PC4 microcon
5
6 while True:
7     # The ADC has 12-bits of re
8     print(adc.read())
9     print("ADC = %fv" % ((adc.re
10    time.sleep_ms(100)
```

With Arduino IDE

Nicla boards ADC can be configured to 8, 10 or 12 bits defining the argument of the following function respectively (default is 10 bits):

```
1 analogReadResolution(12); // AD
```

Example code:

```
1 int potPin = A0; // select the
2
3 void setup() {
4     Serial.begin(115200);
5     analogReadResolution(12);
6 }
7
8 void loop() {
9     // read the value from the ser
10    Serial.print("ADC = ");
11    Serial.print((analogRead(potP
12    Serial.println(" v");
13    delay(100);
14 }
```

The Nicla Vision has **ten digital pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping
PG_12	D0
PA_9	D1
PA_10	D2
PG_1	D3
PE_12	SCKL
PE_13	CIPO
PE_14	COPI
PE_11	CS
PB_8	SCL
PB_9	SDA

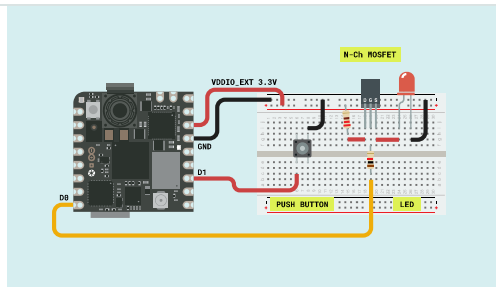
Notice that I2C and SPI pins can also be used as digital pins. Please, refer to the [board pinout section](#) of the user manual to find them on the board.

 The **analog inputs** of the Nicla Vision can be used as digital pins but they can just handle 1.8 V, a greater input may damage the board.

The digital pins of the Nicla Vision can be used as inputs or outputs through the built-in functions of the Arduino programming language.

 The Nicla Vision digital I/O's are low power, so to drive output devices like LEDs, resistive loads, buzzers, etc, it is recommended to use a MOSFET driver or a buffer to guarantee the required current flow. Learn more about the Nicla I/O's considerations [here](#).

As an application example after learning the **Digital Pins** basics, we are going to control an LED using a push button. See the wiring below:



With OpenMV

The configuration of a digital pin is done in the upper section of the code as shown below:

```
1 # Pin configured as an input
2 pin1 = Pin("D1", Pin.IN, Pin.PULL_UP)
3 # Pin configured as an input, in
4 pin1 = Pin("D1", Pin.IN, Pin.PULL_UP)
5 # Pin configured as an output
6 pin0 = Pin("D0", Pin.OUT_PP, Pin.PULL_UP)
```

The pin function can be set as: `Pin.IN`, `Pin.OUT_PP`, `Pin.OUT_OD`, `Pin.AF_PP`, or `Pin.AF_OD`. An explanation of the pin modes can be found [here](#). The third parameter represents the pull-up/pull-down resistor. It can be set to: `Pin.PULL_NONE`, `Pin.PULL_UP` or `Pin.PULL_DOWN`.

The state of a digital pin, configured as an input, can be read as shown below:

```
1 # Reads pin1 state, stores value
2 state = pin1.value()
```

The state of a digital pin, configured as an output, can be changed as shown below:

```
1 # Set pin0 on
2 pin0.value(True)
3 # Set pin0 off
4 pin0.value(False)
```

The example code shown below uses digital pin `D0` to control an LED and reads the state of a button connected to digital pin `D1`:

```
1 import time
2 from machine import Pin
3
4 # Define button and LED pin
5 button = Pin("D1", Pin.IN, Pin.PULLUP)
6 led = Pin("D0", Pin.OUT_PP, Pin.PULLUP)
7
8 while True:
9     if button.value() == 0: #
10         led.value(1)
11         print("- Button is pressed")
12     else: #
13         led.value(0)
14         print("- Button is not pressed")
15     time.sleep_ms(1000) #
```

With Arduino IDE

The configuration of a digital pin is done in the `setup()` function with the built-in function `pinMode()` as shown below:

```
1 // Pin configured as an input
2 pinMode(D1, INPUT);
3 // Pin configured as an input, internal pull-up resistor
4 pinMode(D1, INPUT_PULLUP);
5 // Pin configured as an output
6 pinMode(D0, OUTPUT);
```

The state of a digital pin, configured as an input, can be read using the built-in function `digitalRead()` as shown below:

```
1 // Reads pin state, stores value
2 state = digitalRead(D1);
```

The state of a digital pin, configured as an output, can be changed using the built-in

```

1 // Set pin on
2 digitalWrite(D0, HIGH);
3 // Set pin off
4 digitalWrite(D0, LOW);

```

The example code shown below uses digital pin `D0` to control an LED and reads the state of a button connected to digital pin `D1`:

```

1 // Define button and LED pin
2 int buttonPin = D1;
3 int ledPin = D0;
4
5 // Variable to store the button state
6 int buttonState = 0;
7
8 void setup() {
9   // Configure button and LED
10  pinMode(buttonPin, INPUT_PULLUP);
11  pinMode(ledPin, OUTPUT);
12  // Initialize Serial communication
13  Serial.begin(115200);
14 }
15
16 void loop() {
17   // Read the state of the button
18   buttonState = digitalRead(buttonPin);
19   // If the button is pressed,
20   if (buttonState == LOW) {
21     digitalWrite(ledPin, HIGH);
22     Serial.println("- Button pressed");
23   } else {
24     // If the button is not pressed,
25     digitalWrite(ledPin, LOW);
26     Serial.println("- Button not pressed");
27   }
28   // Wait for 1000 milliseconds
29   delay(1000);

```

PWM Pins

Most digital pins of the Nicla Vision can be used as PWM (Pulse Width Modulation) pins, including I2C and SPI interfaces.

With OpenMV

PWM outputs can be controlled with MicroPython easily by using built-in functions as shown below:

First, we need to identify the `Timer` and `Channel` used by the `PWM` output to be used. For this, search for the desired pin on the [STM32H747 datasheet](#) from page 89.

Here is a table with the details of the exposed pins on the Nicla Vision:

Microcontroller Pin	Arduino Pin Mapping	Timer	Channel
PA_9	D1	TIMER1	CH2
PA_10	D2	TIMER1	CH3
PB_8	SCL	TIMER4	CH3
PB_9	SDA	TIMER4	CH4
PE_11	CS	TIMER1	CH2
PE_12	SCKL	TIMER1	CH3
PE_13	CIPO	TIMER1	CH3
PE_14	COPI	TIMER1	CH4

To use the PWM functions, you need to import the `time`, `Pin`, and `Timer` modules.

```
1 import time
2 from pyb import Pin, Timer
```

First you need to choose the pin you want to use PWM with.

```
1 OUT = Pin("D1", Pin.OUT_PP, Pin.MODE_WO)
```

Create a timer for the PWM, where you set the `timer number` (following the table from above) and the frequency.

```
1 timer1 = Timer(1, freq=1000)
```

timer object.

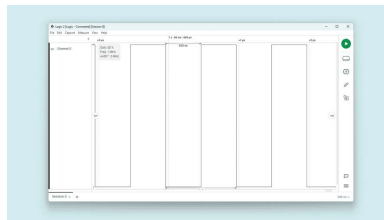
```
1 channel1 = timer1.channel(2, Timer
```

Get or set the pulse width value on a channel.
To get, pass no arguments. To set, give a value as an argument.

```
1 channel1.pulse_width_percentage()
```

As a complete example here is a code to generate a **50% duty cycle** PWM signal at **10 Khz**.

```
1 import time
2 from pyb import Pin, Timer
3
4 # D1 is connected to TIMER1_CH2
5
6 OUT = Pin("D1")
7
8 timer1 = Timer(1, freq=10000)
9 channel1 = timer1.channel(2, Tim
10
11 channel1.pulse_width_percent(50)
12
13
14 while True:
15     time.sleep_ms(1000)
```



PWM output signal

With Arduino IDE

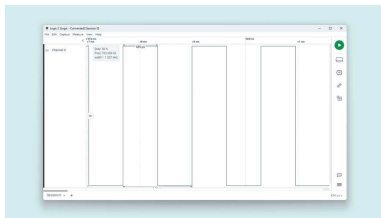
```
1 analogWrite(pin, value);
```

The output resolution is 8 bits by default, so the output value should be between 0 and 255. To set a greater resolution, you can use the built-in function `analogWriteResolution` as shown below:

```
1 analogWriteResolution(bits);
```

Using `analogWrite` has some limitations, for example, the PWM signal frequency is fixed at 763 Hz, and this could not be ideal for every application.

```
1 // 12 bits PWM 50% duty cycle ex-
2
3 void setup() {
4   analogWriteResolution(12);
5 }
6
7 void loop() {
8   analogWrite(D1, 2048);    // P
9 }
```



PWM output signal

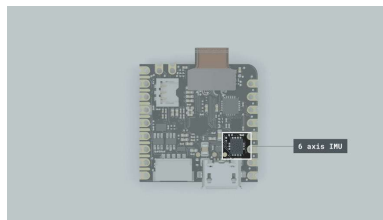
Onboard Sensors

The Nicla Vision comes with various onboard sensors that allow you to capture and process motion data via a 6-axis IMU, distance with a Time of Flight (ToF) sensor, record sound with

The onboard sensors can be used for developing various applications, such as voice commanded projects, activity recognition, vibration detection and image classification. The onboard sensors are suitable for Machine Learning applications using our [Arduino Cloud ML Tools](#).

IMU

The Nicla Vision features an advanced IMU, which allows the board to sense motion. The IMU on the board is the LSM6DSOXTR from ST®. It consists of a 3-axis accelerometer and a 3-axis gyroscope. They can provide information about the board's motion, orientation, and rotation in a 3D space.

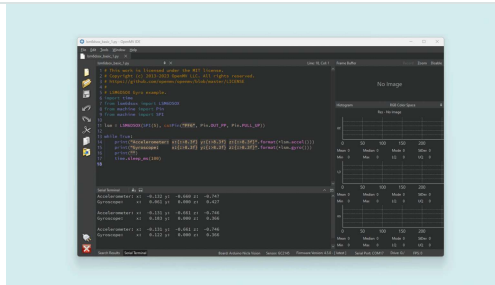


Nicla Vision onboard IMU

With OpenMV

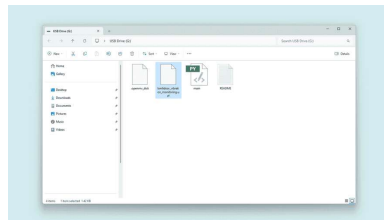
In this MicroPython environment, you can choose between a basic usage of the IMU by sampling raw motion data using the example code below.

```
1 import time
2 from lsm6dsiox import LSM6DSOX
3 from machine import Pin
4 from machine import SPI
5
6 lsm = LSM6DSOX(SPI(5), cs=Pin("P1"))
7
8 while True:
9     print("Accelerometer: x:{:>6f} y:{:>6f} z:{:>6f}"
10         .format(lsm.accelerometer.get_x(), lsm.accelerometer.get_y(), lsm.accelerometer.get_z()))
11     print("Gyroscope: x:{:>6f} y:{:>6f} z:{:>6f}"
12         .format(lsm.gyroscope.get_x(), lsm.gyroscope.get_y(), lsm.gyroscope.get_z()))
13     print("")
14     time.sleep_ms(100)
```



Also, you can develop Machine Learning applications using the Nicla Vision IMU. As a practical example, we are going to test a `Vibration monitoring` model that will be able to identify three states: `no vibration`, `low vibration` and `high vibration`.

First, download the pre-trained model file from the [example repository](#) and copy it to the Nicla Vision storage drive.



UCF Machine Learning model in
the drive

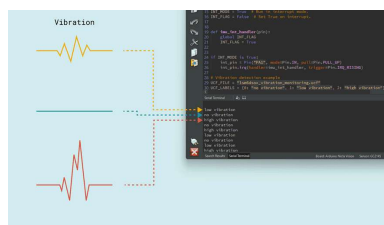
Reset the board and run the following code on the OpenMV IDE.

```

1 from machine import Pin
2 from machine import SPI
3 from lsm6dsox import LSM6DSOX
4
5 INT_MODE = True # Run in inte
6 INT_FLAG = False # Set True o
7
8
9 def imu_int_handler(pin):
10     global INT_FLAG
11     INT_FLAG = True
12
13
14 if INT_MODE is True:
15     int_pin = Pin("PA1", mode=
16     int_pin.irq(handler=imu_in
17
18 # Vibration detection example
19 UCF_FILE = "lsm6dsox_vibration
20 UCF_LABELS = {0: "no vibration
21 # NOTE: Selected data rate and
22 lsm = LSM6DSOX(
23     SPI(5),
24     cs=Pin("PF6", Pin.OUT_PP,
25     gyro_odr=26,
26     accel_odr=26,
27     gyro_scale=2000,
28     accel_scale=4,

```

In the OpenMV IDE Serial Monitor, the inference results will be printed after a vibration event.



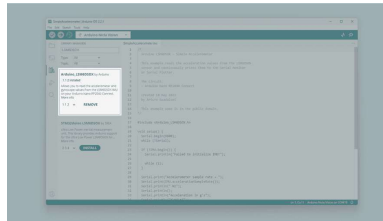
Model inference results

 You can download and test many other pre-trained models available in this [repository](#).

With Arduino IDE

First, to use this sensor with the Arduino IDE, you need to install the `Arduino_LSM6DSOX` library, which can be found in the Arduino IDE library manager. To do so in the IDE, select it

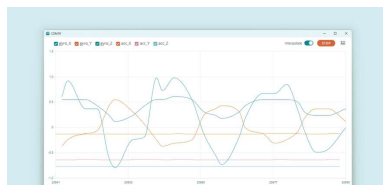
from the left side menu, search for `LSM6DSOX` and install the one from Arduino.



IMU library installation

The example code below shows how to get acceleration and angular velocity data from the onboard IMU and stream it to the IDE's Serial Monitor and Serial Plotter.

```
1  #include <Arduino_LSM6DSOX.h>
2
3  void setup() {
4      Serial.begin(9600);
5      while (!Serial)
6          ;
7
8      if (!IMU.begin()) {
9          Serial.println("Failed to
10
11          while (1)
12              ;
13      }
14
15      Serial.print("Accelerometer
16      Serial.print(IMU.acceleratio
17      Serial.println(" Hz");
18      Serial.println();
19      Serial.println("Acceleration
20      Serial.println("X\tY\tZ");
21
22      Serial.print("Gyroscope samp
23      Serial.print(IMU.gyroscopeSa
24      Serial.println(" Hz");
25      Serial.println();
26      Serial.println("Gyroscope in
27      Serial.println("X\tY\tZ");
28
29      while (1) {
30          // Read raw acceleration data (longitudinal)
```



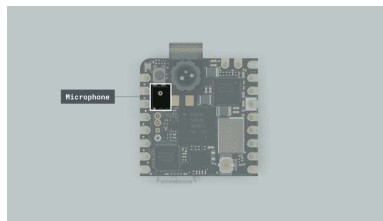
Accelerometer and gyroscope output in the serial plotter

To test a Machine Learning model on the Arduino IDE, navigate to **File > Examples > MLC >**

i **NiclaVision_MLC_Motion_Intesity** and it will identify three scenarios: **Stationary**, **Medium Intensity** and **High Intensity** movements.

Microphone

The onboard high-performance microphone of the Nicla Vision is the MP34DT06JTR from ST®. It is specifically designed for applications that require high-quality audio recording and accurate voice detection, such as voice-controlled Internet of Things (IoT) devices, smart home systems, and mobile devices.

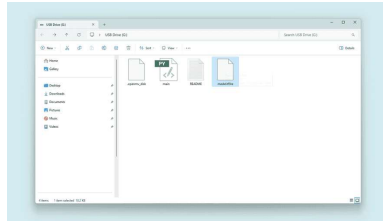


Nicla Vision onboard high-performance microphone

Using OpenMV

The OpenMV IDE includes some examples to get started using the Nicla Vision onboard microphone that can be found on **File > Examples > Audio**. We are going to use the one called `micro_speech.py` to test the machine-learning speech recognition capabilities of the board.

First, download the pre-trained model file from the [example repository](#) and **copy** it to the Nicla Vision **storage drive**.



Tflite Machine Learning model in the drive

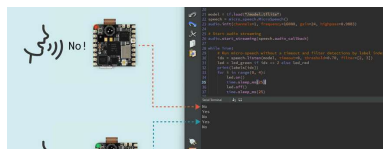
Reset the board and run the following code on the OpenMV IDE.

```

1 import time
2 from ml.apps import MicroSpeech
3 import pyb
4
5
6 led_red = pyb.LED(1)
7 led_green = pyb.LED(2)
8
9
10 def callback(label, scores):
11     print(f'\nHeard: "{label}"')
12
13     led = led_green if label ==
14
15     for i in range(0, 4):
16         led.on()
17         time.sleep_ms(25)
18         led.off()
19         time.sleep_ms(25)
20
21
22
23
24 # By default, the MicroSpeech
25 # micro speech module for audi
26 # user can override both by pa
27 # MicroSpeech(preprocessor=ml.
28 speech = MicroSpeech()

```

After running the code, the matches will be printed on the Serial Monitor if the board hears a or a , turning on the red and green LED respectively.



Speech recognition inference results

Using Arduino IDE

The Arduino IDE includes a simple example to visualize raw data from the PDM microphone. To test it, navigate to **File > Examples > PDM > PDMSerialPlotter**.

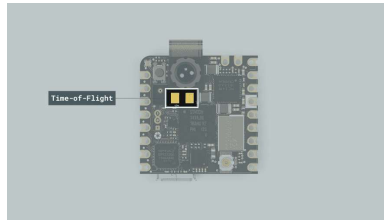
```
1  #include <PDM.h>
2
3  // default number of output ch
4  static const char channels = 1
5
6  // default PCM output frequenc
7  static const int frequency = 1
8
9  // Buffer to read samples into
10 short sampleBuffer[512];
11
12 // Number of audio samples rea
13 volatile int samplesRead;
14
15 void setup() {
16   Serial.begin(9600);
17   while (!Serial);
18
19   // Configure the data receiv
20   PDM.onReceive(onPDMdata);
21
22   // Optionally set the gain
23   // Defaults to 20 on the BLE
24   // PDM.setGain(30);
25
26   // Initialize PDM with:
27   // - one channel (mono mode)
28   // - a 16 kHz sample rate fo
```

Upload the example code to the Nicla Vision and open the Serial Plotter to see the sound wave output.



Time of Flight (Distance) Sensor

The onboard ToF sensor of the Nicla Vision is the VL53L1CBV0FY from ST®. It adds accurate and low power ranging capabilities to the Nicla Vision. The invisible near-infrared VCSEL laser (including the analog driver) is encapsulated with receiving optics in an all-in-one small module located below the camera.



Onboard Time-of-Flight ranging sensor

Here are listed the sensor's main features:

- Up to 400 cm distance measurement

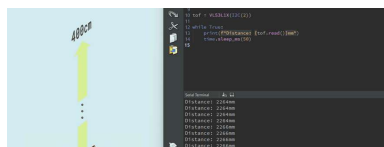
- Up to 50 Hz ranging frequency

- 27° field-of-view (FoV)

With OpenMV

The OpenMV IDE includes an example to start using the ToF sensor. To test it, navigate to **File > Examples > Sensors > vl53l1x_tof** and run it on the Nicla Vision.

```
1 from machine import I2C
2 from vl53l1x import VL53L1X
3 import time
4
5 tof = VL53L1X(I2C(2))
6
7 while True:
8     print(f"Distance: {tof.read(
9         time.sleep_ms(50))
```



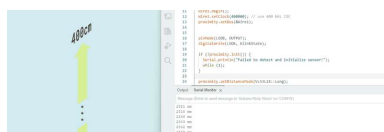
With Arduino IDE

To use the ToF sensor with the Arduino IDE, install the `VL53L1X` library authored by Pololu by searching for it on the IDE library manager and clicking on install.

Once installed, you will be able to compile and upload the example code below to your Nicla Vision.

The distance measured by the sensor will be printed on the IDE's Serial Monitor, and the built-in LED will blink proportionally to that distance.

```
1  #include "VL53L1X.h"
2  VL53L1X proximity;
3
4  bool blinkState = false;
5  int reading = 0;
6  int timeStart = 0;
7  int blinkTime = 2000;
8
9  void setup() {
10   Serial.begin(115200);
11   Wire1.begin();
12   Wire1.setClock(400000); // u
13   proximity.setBus(&Wire1);
14
15
16   pinMode(LED_B, OUTPUT);
17   digitalWrite(LED_B, blinkStat
18
19   if (!proximity.init()) {
20     Serial.println("Failed to
21     while (1);
22   }
23
24   proximity.setDistanceMode(VL
25   proximity.setMeasurementTimi
26   proximity.startContinuous(50
27 }
28
```



Camera

The Nicla Vision's main feature is its onboard 2MP camera, based on the GC2145 color rolling shutter image sensor. It is perfect for Machine Learning applications such as object detection, image classification, machine/computer vision, robotics, IoT, and more.



Onboard camera sensor

The Nicla Vision is primarily intended to be used with the OpenMV MicroPython ecosystem. So, it's recommended to use this IDE for machine vision applications.

With OpenMV

The OpenMV IDE is designed to work specifically with machine/computer vision hardware, it is optimized for easy and fast development of image processing applications with a MicroPython framework and streaming monitors, color data graphics, and more.

The Nicla Vision uses a 2MP camera sensor, meaning its maximum resolution is 1920x1080 pixels. However, the effective resolution is 1616(H) × 1232(V).

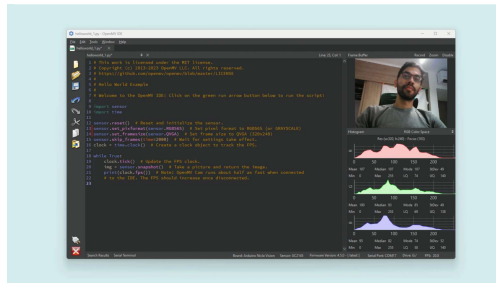
Here we have the minimum code necessary to make the camera work streaming live video on the OpenMV IDE:



```

1 import sensor
2 import time
3
4 sensor.reset() # Reset and initialize
5 sensor.set_pixformat(sensor.RGB565)
6 sensor.set_framesize(sensor.QVGA)
7 sensor.skip_frames(time=2000) # Skip
8 clock = time.clock() # Create a clock
9
10 while True:
11     clock.tick() # Update the clock
12     img = sensor.snapshot() # Take a
13     print(clock.fps()) # Note:
14     # to the IDE. The FPS should be

```



From the above example script, we can highlight the main functions:

```
sensor.set_pixformat(<Sensor>)
```

lets you set the pixel format for the camera sensor. The Nicla Vision is compatible with these:

```
sensor.GRAYSCALE, sensor.RGB565,
sensor.BAYER, and sensor.YUV422.
```

To define the pixel format to any of the supported ones, just add it to the `set_pixformat` function argument.

```
sensor.set_framesize(<Resolution>)
```

lets you define the image frame size in terms of pixels. [Here](#) you can find all the different options.

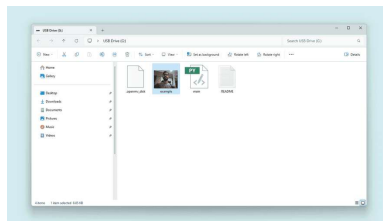
```
sensor.snapshot()
```

lets you take a picture and return the image so you can save it, stream it or process it.

The example code below lets you take a picture and save it on the Nicla Vision local storage as `example.jpg`.

```
1 import sensor
2 import time
3 import machine
4
5 sensor.reset() # Reset and init
6 sensor.set_pixformat(sensor.RGB565)
7 sensor.set_framesize(sensor.QVGA)
8 sensor.skip_frames(time=2000)
9
10 led = machine.LED("LED_BLUE")
11
12 start = time.ticks_ms()
13 while time.ticks_diff(time.ticks_ms(), start) < 1000:
14     sensor.snapshot()
15     led.toggle()
16
17 led.off()
18
19 img = sensor.snapshot()
20 img.save("example.jpg") # or "example.mjpg"
21
22 raise Exception("Please reset the board")
```

After the snapshot is taken, reset the board by pressing the reset button and the image will be on the board storage drive.

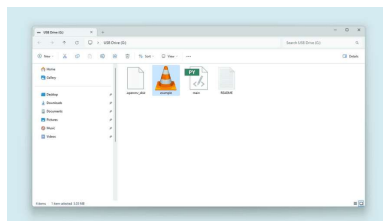


Snapshot saved in local storage

The example code below lets you record a video and save it on the Nicla Vision local storage as `example.mjpeg`.

```
1 import sensor
2 import time
3 import mjpeg
4 import machine
5
6 sensor.reset() # Reset and init
7 sensor.set_pixformat(sensor.RGB565)
8 sensor.set_framesize(sensor.QVGA)
9 sensor.skip_frames(time=2000)
10
11 led = machine.LED("LED_RED")
12
13 led.on()
14 m = mjpeg.Mjpeg("example.mjpeg")
15
16 clock = time.clock() # Create a clock
17 for i in range(200):
18     clock.tick()
19     m.add_frame(sensor.snapshot())
20     print(clock.fps())
21
22 m.close(clock.fps())
23 led.off()
24
25 raise Exception("Please reset the board")
```

After the video is recorded, reset the board by pressing the reset button and the file will be on the board storage drive.



Video saved in local storage



We recommend using **VLC** to play the video due to the format.

The next example lets you live stream what the camera sees through HTTP so you can watch it on your favorite browser from any device connected to the same network as the Nicla Vision.

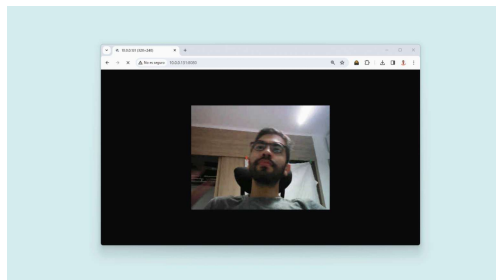
Make sure to fill in the `SSID` and `KEY` variables with your Wi-Fi® credentials.

```
1 import sensor
2 import time
3 import network
4 import socket
5
6 SSID = "*****" # Network
7 KEY = "*****" # Networ
8 HOST = "" # Use first availab
9 PORT = 8080 # Arbitrary non-p
10
11 # Init sensor
12 sensor.reset()
13 sensor.set_framesize(sensor.QV
14 sensor.set_pixformat(sensor.RG
15
16 # Init wlan module and connect
17 wlan = network.WLAN(network.ST
18 wlan.active(True)
19 wlan.connect(SSID, KEY)
20
21 while not wlan.isconnected():
22     print('Trying to connect t
23     time.sleep_ms(1000)
24
25 # We should have a valid IP no
26 print("WiFi Connected ", wlan.
27
28 # Create server socket
```

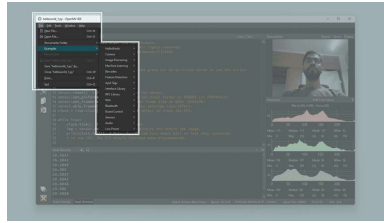
Once you run this script, the Nicla Vision IP address will be printed on the OpenMV serial monitor after the Wi-Fi® connection processes.

To watch the live stream, enter the device IP address followed by the `:8080` port as follows:

```
<Nicla Vision IP>:8080
```



To expand your knowledge using the Nicla Vision camera with MicroPython, try other built-in examples within the OpenMV IDE.



OpenMV examples for the Nicla
Vision

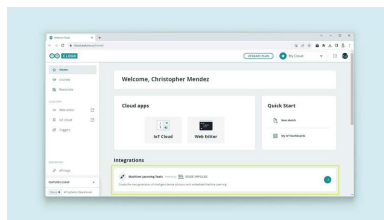
Machine Learning Tools

The Nicla Vision is a ready-to-use, standalone camera board, ready for analyzing and processing images on the Edge. Thanks to its 2MP color camera, smart 6-axis motion sensor, integrated microphone, and distance sensor, it is suitable for almost infinite machine-learning applications.

Creating this type of application has never been easier thanks to our Machine Learning Tool powered by Edge Impulse®, where we can easily create in a **No-Code** environment, **Audio, Motion, Proximity** and **Image** processing models.

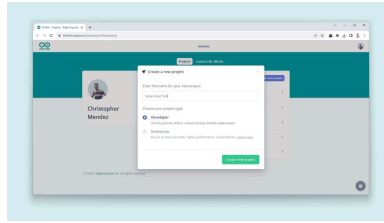
The first step to start creating awesome artificial intelligence and machine learning projects is to create an [Arduino Cloud](#) account.

There you will find a dedicated integration called **Machine Learning Tools**.



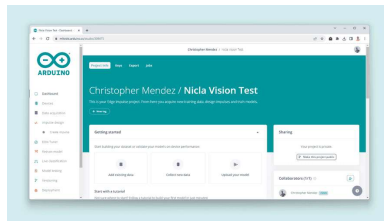
Machine Learning Tools on Arduino
Cloud

Once in, create a new project and give it a name.



Creating a new project

Enter your newly created project and the landing page will look like the following:



Nicla Vision project page

Edge Impulse® Environment Setup

Now, it is time to set up the **Edge Impulse®** environment on your PC. For this, follow [these](#) instructions to install the **Edge Impulse CLI**.

For Windows users: make sure to install  **Visual Studio Community** and **Visual Studio Build Tools**.

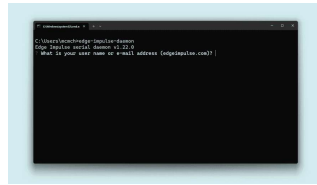
Download and install the latest **Arduino CLI** from [here](#). ([Video Guide for Windows](#))

Download the **latest Edge Impulse® firmware**, and unzip the file.

Open the flash script for your operating system (`flash_windows.bat`, `flash_mac.command` or `flash_linux.sh`) to flash the firmware.

To test if the **Edge Impulse CLI** was installed correctly, open the **Command Prompt** or your favorite terminal and

If everything went okay, you should be asked for your account credentials.

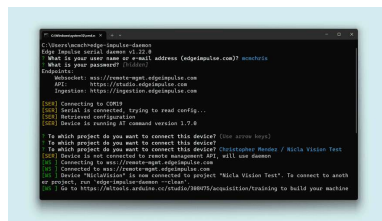


Edge Impulse Daemon

Enter your account username or e-mail address and your password.

Select the project you have created on the Arduino ML Tools, it will be listed.

Give your device a name and wait for it to connect to the platform.

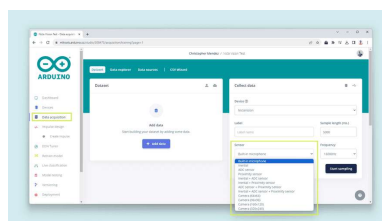


Nicla Vision correctly connected to ML Tools

Uploading Sensor Data

The first thing to start developing a machine learning project is to create a dataset for your model. This means, uploading data to your model from any of the Nicla Vision sensors.

To upload data from your Nicla Vision on the Machine Learning Tools platform, navigate to **Data Acquisition**.



Data Acquisition section

In this section, you will be able to select the

This is the supported sensors list:

- Built-in microphone
- Inertial (IMU)
- ADC sensor (A0)
- Proximity sensor
- Camera (different resolutions)

Now you know how to start with our **Machine Learning Tools** creating your dataset from scratch, you can get inspired by some of our ML projects listed below:

[Image Classification with Edge Impulse®](#) (Article).

[Glass-Breaking Detector using Edge Impulse®](#) (Video).

Communication

This section of the user manual covers the different communication protocols that are supported by the Nicla Vision, including the Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I2C), Universal Asynchronous Receiver-Transmitter (UART), and Bluetooth® Low Energy; communication via the onboard ESLOV connector is also explained in this section. The Nicla Vision features dedicated pins for each communication protocol, making connecting and communicating with different components, peripherals, and sensors easy.

SPI

The Nicla Vision supports SPI communication, which allows data transmission between the board and other SPI-compatible devices. The pins used in the Nicla Vision for the SPI communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
SCLK / PE_12	SCKL or PE12
CIPO / PE_13	CIPO or PE13
COPI / PE_14	COPI or PE14

Please, refer to the [board pinout section](#) of the user manual to localize them on the board.

With OpenMV

Import the `SPI` submodule from the `pyb` module at the top of your sketch alongside `time` and `Pin` to use the SPI communication protocol. The SPI driver provides functions for SPI communication:

```
1 import time
2 from pyb import Pin, SPI
```

Before your infinite loop, configure the chip select (`CS`) pin and initialize the SPI peripheral as a `Master`:

```
1 spi = SPI(4, SPI.MASTER, baudrate=
```

To send data over SPI, use:

```
1 spi.send(<data>) # add the data
```

To receive data over SPI, use:

```
1 spi.recv(<bytes>) # add the number
```

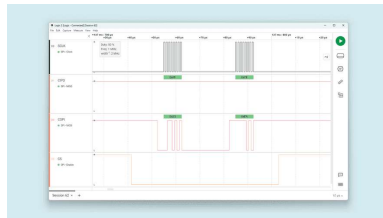
Here is a simple example showing how to send data over SPI.

```

1 import time
2 from pyb import Pin, SPI
3
4 cs = Pin("CS", Pin.OUT_OD) # CS
5
6 spi = SPI(4, SPI.MASTER, baudrat
7
8 while True:
9     # Replace with the target d
10    address = 0x35
11    # Replace with the value to
12    value = 0xFA
13    # Pull the CS pin LOW to se
14    cs.low()
15    # Send the address
16    spi.send(address)
17    # Send the value
18    spi.send(value)
19    # Pull the CS pin HIGH to u
20    cs.high()
21
22    time.sleep_ms(1000)

```

The example code above should output this:



SPI logic data output

With Arduino IDE

Include the `SPI` library at the top of your sketch to use the SPI communication protocol. The SPI library provides functions for SPI communication:

```
1 #include <SPI.h>
```

In the `setup()` function, initialize the SPI library, define and configure the chip select (`cs`) pin:

```
1 void setup() {
2   // Set the chip select pin as output
3   pinMode(SS, OUTPUT);
4   // Pull the CS pin HIGH to unselect device
5   digitalWrite(SS, HIGH);
6
7   // Initialize the SPI communication
8   SPI.begin();
9 }
```

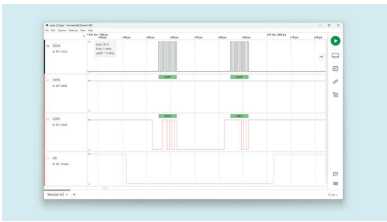
To transmit data to an SPI-compatible device, you can use the following commands:

```
1 // Replace with the target device's address
2 byte address = 0x35;
3 // Replace with the value to send
4 byte value = 0xFA;
5 // Pull the CS pin LOW to select device
6 digitalWrite(SS, LOW);
7 // Send the address
8 SPI.transfer(address);
9 // Send the value
10 SPI.transfer(value);
11 // Pull the CS pin HIGH to unselect device
12 digitalWrite(SS, HIGH);
```

Here is the complete sketch for a simple SPI communication:

```
1 #include <SPI.h>
2
3 void setup(){
4     // Set the chip select pin as
5     pinMode(SS, OUTPUT);
6     // Pull the CS pin HIGH to uns
7     digitalWrite(SS, HIGH);
8
9     // Initialize the SPI communic
10    SPI.begin();
11 }
12
13 void loop(){
14     // Replace with the target dev
15     byte address = 0x35;
16     // Replace with the value to s
17     byte value = 0xFA;
18     // Pull the CS pin LOW to sele
19     digitalWrite(SS, LOW);
20     // Send the address
21     SPI.transfer(address);
22     // Send the value
23     SPI.transfer(value);
24     // Pull the CS pin HIGH to uns
25     digitalWrite(SS, HIGH);
26     delay(1000);
27 }
```

The example code above should output this:



SPI logic data output

I2C

The Nicla Vision supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. The pins used in the Nicla Vision for the I2C communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
PB_8 / SCL	SCL / I2C1_SCL / PB8
PB_9 / SDA	SDA / I2C1_SDA / PB9

Please, refer to the [board pinout section](#) of the user manual to localize them on the board. The I2C pins are also available through the onboard ESLOV connector of the Nicla Vision.

With OpenMV

To use I2C communication with OpenMV, import `I2C` from the `machine` module as follows.

```
1 from machine import I2C
```

Create the I2C object and initialize it attached to defined pins, below is the available command to do it.

```
1 i2c = I2C(scl="PB8", sda="PB9", fr
```

The basic methods are `writeto` and `readfrom` implemented as follows:

```
1 i2c.writeto(42, b'123')
2
3 i2c.readfrom(42, 4)
```

You can scan for peripherals with the following function:

```
1 i2c.scan()
```

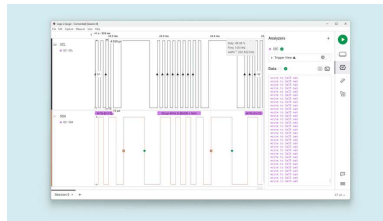
This is a simple example showing how to send data over I2C from a master to a slave.

```

1 from machine import I2C
2
3 i2c = I2C(scl="PB8", sda="PB9", fr
4
5 buf = bytearray(2)
6 buf[0] = 0x00
7 buf[1] = 0xFA
8
9 i2c.writeto(0x35, buf)

```

The output data should look like the image below, where we can see the device address data frame:



I2C output data

To learn more about the I2C class on MicroPython, continue [here](#).

With Arduino IDE

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```

1 #include <Wire.h>

```

In the `setup()` function, initialize the I2C library:

```

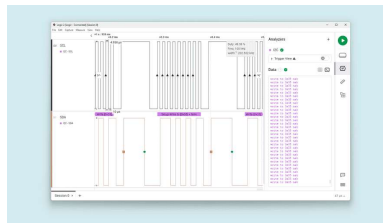
1 // Initialize the I2C communicat
2 Wire.begin();

```

To transmit data to an I2C-compatible device, you can use the following commands:


```
1 // Replace with the target device
2 byte deviceAddress = 0x35;
3 // Replace with the appropriate
4 byte instruction = 0x00;
5 // Replace with the value to send
6 byte value = 0xFA;
7 // Begin transmission to the target
8 Wire.beginTransmission(deviceAddress);
9 // Send the instruction byte
10 Wire.write(instruction);
11 // Send the value
12 Wire.write(value);
13 // End transmission
14 Wire.endTransmission();
```

The output data should look like the image below, where we can see the device address data frame:



I2C output data

Here is the complete sketch for a simple I2C communication:

```

1  #include <Wire.h>
2
3  void setup() {
4      // Initialize the I2C communica
5      Wire.begin();
6  }
7
8  void loop() {
9      // Replace with the target dev
10     byte deviceAddress = 0x35;
11     // Replace with the appropriat
12     byte instruction = 0x00;
13     // Replace with the value to s
14     byte value = 0xFA;
15     // Begin transmission to the t
16     Wire.beginTransmission(device
17     // Send the instruction byte
18     Wire.write(instruction);
19     // Send the value
20     Wire.write(value);
21     // End transmission
22     Wire.endTransmission();
23     delay(1000);
24 }

```

To read data from an I2C-compatible device, you can use the `requestFrom()` function to request data from the device and the `read()` function to read the received bytes:

```

1  // The target device's I2C addre
2  byte deviceAddress = 0x1;
3  // The number of bytes to read
4  int numBytes = 2;
5  // Request data from the target
6  Wire.requestFrom(deviceAddress,
7  // Read while there is data ava
8  while (Wire.available()) {
9      byte data = Wire.read();
10 }

```

UART

The pins used in the Nicla Vision for the UART (external) communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping

Microcontroller Pin	Arduino Pin Mapping
PA_9	UART_TX or PA9

Please, refer to the [board pinout section](#) of the user manual to localize them on the board.

With OpenMV

To begin with UART communication, you will need to import `UART` from the `machine` module.

```
1 from machine import UART
```

Then, initialize the UART object defining the **bus number** and **baudrate**.

```
1 uart = UART(9, 115200) # bus 9
```

You can define UART settings with the following function:

```
1 uart.init(9600, bits=8, parity=N
```

To read incoming data, you can use different functions as the following.

```
1 uart.read(10)      # read 10 ch
2 uart.read()        # read all av
3 uart.readline()    # read a line
4 uart.readinto(buf) # read and s
```

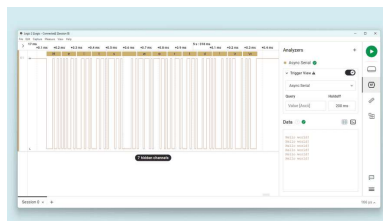
To write data, use the following function.

```
1 uart.write('abc') # write the
```

Here is the complete example that writes "Hello World!" on the external serial port of the Nicla Vision.

```
1 import time
2 from machine import UART
3
4 # Init UART object.
5 uart = UART(9, 115200)
6
7 while True:
8     uart.write("Hello World!\r")
9     time.sleep_ms(1000)
```

This is the output of the example code from above.



UART "Hello World!" data frame

With Arduino IDE

To begin with UART communication, you will need to configure it first. In the `setup()` function, set the baud rate (bits per second) for UART communication:

```
1 // Start UART communication at 115200
2 Serial1.begin(115200); // Serial
```



Using the Arduino IDE, the minimum supported baud rate is 19200.

available data and read individual characters. The code shown above stores the incoming characters in a String variable and processes the data when a line-ending character is received:

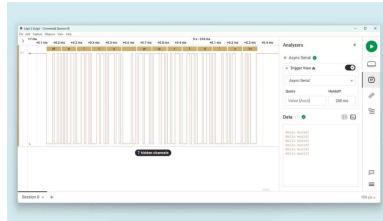
```
1 // Variable for storing incoming
2 String incoming = "";
3 void loop() {
4   // Check for available data and
5   while (Serial1.available()) {
6     // Allow data buffering and
7     delay(2);
8     char c = Serial1.read();
9
10    // Check if the character is
11    if (c == '\n') {
12      // Process the received data
13      processData(incoming);
14      // Clear the incoming data
15      incoming = "";
16    } else {
17      // Add the character to the
18      incoming += c;
19    }
20  }
21 }
```

To transmit data to another device via UART, you can use the `write()` function:

```
1 // Transmit the string "Hello world!"
2 Serial1.write("Hello world!");
```

You can also use the `print` and `println()` to send a string without a newline character or followed by a newline character:

```
1 // Transmit the string "Hello world!"
2 Serial1.print("Hello world!");
3 // Transmit the string "Hello world!"
4 Serial1.println("Hello world!");
```



UART "Hello World!" data frame

i If you want to communicate through the USB serial port, use "Serial" instead of "Serial1" for this case.

Bluetooth® Low Energy

To enable the Bluetooth® Low Energy communication on the Nicla Vision, you can use the `bluetooth` module in [OpenMV](#) or the [ArduinoBLE library](#) in the [Arduino IDE](#).

We are going to build a Bluetooth® LE temperature monitor that using the **nRF Connect** app (available for [Android](#) and [iOS](#)) will let us easily connect to our Nicla Vision and monitor the temperature in real time.

With OpenMV

For this Bluetooth® LE application example, we are going to emulate the temperature sensor. Below you will find the complete sketch.



```
1 from micropython import const
2 import uasyncio as asyncio
3 import aioble
4 import bluetooth
5 import random
6 import struct
7 # org.bluetooth.service.enviro
8 _ENV_SENSE_UUID = bluetooth.UU
9 # org.bluetooth.characteristic
10 _ENV_SENSE_TEMP_UUID = bluetoo
11 # org.bluetooth.characteristic
12 _ADV_APPEARANCE_GENERIC_THERMO
13 # How frequently to send adver
14 _ADV_INTERVAL_MS = 250_000
15 # Register GATT server.
16 temp_service = aioble.Service(
17     temp_characteristic = aioble.C
18     temp_service, _ENV_SENSE_T
19 )
20 aioble.register_services(temp_
21 # Helper to encode the tempera
22 def _encode_temperature(temp_d
23     return struct.pack("<h", i
24 # This would be periodically p
25 async def sensor_task():
26     t = 24.5
27     while True:
28         temp_characteristic.wr
```

The example code shown above creates a Bluetooth® Low Energy service and characteristics according to the [Bluetooth® LE standard](#) for transmitting an emulated temperature value.

The code begins by importing all the necessary modules and defining the Bluetooth® Low Energy service and characteristics for an environment-sensing application.

Description	ID
Environmental Sensing Service	181A
Temperature Characteristic	2A6E

Then sets up the Bluetooth® Low Energy service and characteristics; and begins advertising the defined Bluetooth® Low Energy service.

A Bluetooth® Low Energy connection is constantly verified; when a central device connects to the Nicla Vision. its

then enters into a loop that constantly emulates a temperature reading.

It also prints it to the Serial Monitor and transmits it to the central device over the defined Bluetooth® Low Energy characteristic.

With Arduino IDE

For this Bluetooth® LE application example, we are going to monitor the **Nicla Vision IMU temperature sensor**. Below you will find the complete sketch.

```

1  #include <ArduinoBLE.h>
2  #include <Arduino_LSM6DSOX.h>
3  // Bluetooth® Low Energy Envi
4  BLEService environmentService
5  // Bluetooth® Low Energy Temp
6  BLEIntCharacteristic temperat
7
8  int oldTemperature = 0;
9
10 long previousMillis = 0; //
11 void blePeripheralDisconnectH
12   digitalWrite(LED_R, LOW); /
13   digitalWrite(LED_G, HIGH);
14   digitalWrite(LED_B, HIGH);
15   Serial.println("Device disc
16 }
17 void blePeripheralConnectHand
18   digitalWrite(LED_B, LOW); /
19   digitalWrite(LED_R, HIGH);
20   digitalWrite(LED_G, HIGH);
21   Serial.println("Device conn
22 }
23 void setup() {
24   Serial.begin(9600); // ini
25   while (!Serial)
26     ;
27
28   pinMode(LED_R, OUTPUT);

```

The example code shown above creates a Bluetooth® Low Energy service and characteristics according to the **Bluetooth® LE standard** for transmitting temperature value read by Nicla Vision IMU IC.

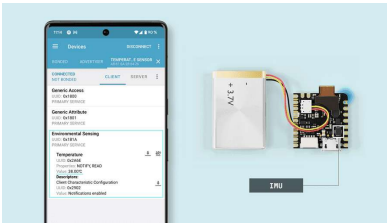
The code begins by importing all the necessary libraries and defining the

characteristics for an environment sensing application.

Description	ID
Environmental Sensing Service	181A
Temperature Characteristic	2A6E

In the `setup()` function, the code initializes the Nicla Vision board and sets up the Bluetooth® Low Energy service and characteristics; then, it begins advertising the defined Bluetooth® Low Energy service.

A Bluetooth® Low Energy connection is constantly verified in the `loop()` function; when a central device connects to the Nicla Vision, its built-in LED is turned on blue. The code then enters into a loop that constantly reads the IMU temperature sensor. It also prints it to the Serial Monitor and transmits it to the central device over the defined Bluetooth® Low Energy characteristic.



Nicla Vision temperature monitored from the nRF Connect app

Wi-Fi®

The Nicla Vision onboard IEEE802.11 b/g/n Wi-Fi® interface can be operated as an access point (AP), station (STA) or dual-mode simultaneous AP/STA. It supports a maximum transfer rate of 65 Mbps.

With OpenMV

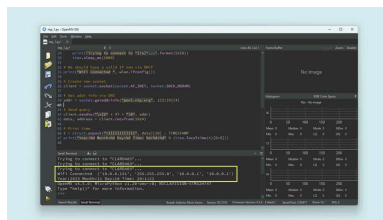
```

1 import network
2 import socket
3 import struct
4 import time
5
6 SSID = "" # Network SSID
7 KEY = "" # Network key
8
9 TIMESTAMP = 2208988800
10
11 if time.gmtime(0)[0] == 2000:
12     TIMESTAMP += 946684800
13
14 # Init wlan module and connect
15 print("Trying to connect... (T
16 wlan = network.WLAN(network.ST
17 wlan.active(True)
18 wlan.connect(SSID, KEY)
19
20 while not wlan.isconnected():
21     print('Trying to connect t
22     time.sleep_ms(1000)
23
24 # We should have a valid IP no
25 print("WiFi Connected ", wlan.
26
27 # Create new socket
28 client = socket.socket(socket.

```

Make sure to enter your Wi-Fi® credentials on the `SSID` and `KEY` variables and run the script from the OpenMV IDE.

The current time and date will be printed on the IDE serial monitor.



Requesting time with NTP server



If you want to learn more about using Nicla Vision's Wi-Fi® with OpenMV, explore the built-in examples on **File > Examples > WiFi**.

The example code below shows how to get the current time using NTP.

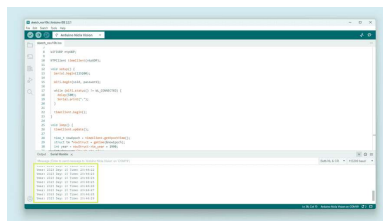
```

1  #include <NTPClient.h> //http
2  #include <WiFi.h>
3  #include <WiFiUdp.h>
4
5  const char *ssid = "";
6  const char *password = "";
7
8  WiFiUDP ntpUDP;
9
10 NTPClient timeClient(ntpUDP);
11
12 void setup() {
13   Serial.begin(115200);
14
15   WiFi.begin(ssid, password);
16
17   while (WiFi.status() != WL_C
18     delay(500);
19     Serial.print(".");
20 }
21
22 timeClient.begin();
23 }
24
25 void loop() {
26   timeClient.update();
27
28   time_t nowEpoch = timeClient

```

Make sure to enter your Wi-Fi® credentials on the `*ssid` and `*password` variables and upload the code from the Arduino IDE.

The current time and date will be printed on the IDE serial monitor.



Requesting time with NTP server

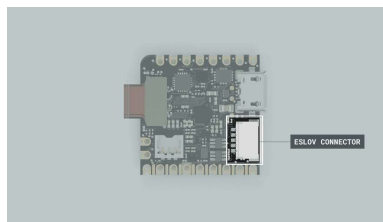


If your Nicla Vision reports an error when trying to connect to Wi-Fi® saying *Failed to mount the filesystem containing the WiFi firmware* follow the next steps

In the Arduino IDE navigate to **File > Examples > STM32H747_System > WiFiFirmwareUpdater**, upload this code to your Nicla Vision and wait for the update. The progress can be followed in the Serial Monitor.

ESLOV Connector

The Nicla Vision board features an onboard ESLOV connector meant as an **extension** of the I2C communication bus. This connector simplifies the communication between the Nicla Vision and various sensors, actuators, and other modules without soldering or wiring.



Nicla Vision built-in ESLOV connector

The ESLOV connector is a small 5-pin connector with a 1.00 mm pitch; the mechanical details of the connector can be found in the connector's datasheet.

The pin layout of the ESLOV connector is the following:

- 1 VCC_IN (5V input)
- 2 INT
- 3 SCL
- 4 SDA
- 5 GND

Arduino Cloud

Leveraging the Nicla Vision's Wi-Fi® connectivity we can develop Smart IoT projects using the [Arduino Cloud](#).

By using the Arduino Cloud, you can, for

remotely, and update your device's firmware OTA.

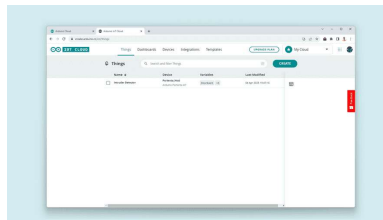
In case it is the first time you are using the Arduino Cloud:

You need an **account**. If you do not have an account, create one for free [here](#).

To use the Arduino Cloud Editor or Arduino Cloud, the **Arduino Create Agent** must be running on your computer. You can install the Arduino Create Agent [here](#).

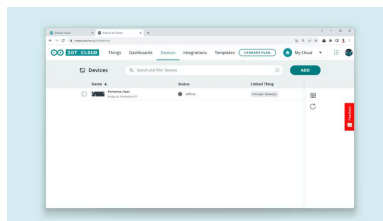
Let's walk through a step-by-step demonstration of how to use a Nicla Vision with the Arduino Cloud.

Log in to your Arduino Cloud account; you should see the following (without any "Thing" created):



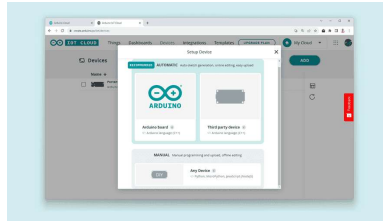
Arduino Cloud initial page

First, provision your Nicla Vision on your Arduino Cloud space. To do this, navigate to **Devices** and then click on the **ADD** button:



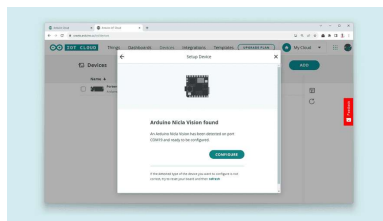
Arduino Cloud Devices Page

The Setup Device pop-up window will appear. Navigate into **AUTOMATIC** and select the **Arduino board** option:



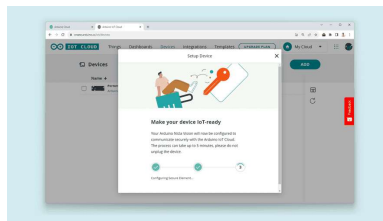
Arduino Cloud Setup Device pop-up window

After a while, your Nicla Vision should be discovered by the Arduino Cloud, as shown below:



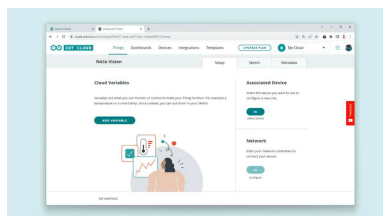
Nicla Vision found and ready to be configured

Click the **CONFIGURE** button, give your device a name, and your Nicla Vision will be configured to communicate securely with the Arduino Cloud; this process can take a while.



Nicla Vision setup process

Once your Nicla Vision has been configured, let's create a **"Thing"** to test the connection between your board and the Arduino Cloud. Navigate into **Things** and select the **CREATE** button; give your thing a name.



Navigate into **Associate Device** and click the **Select Device** button. Select your Nicla Vision and associate it with your "Thing." Then, navigate into **Network** and click the **Configure** button; enter your network credentials.

The project is ready to add variables to your "Thing"; navigate into **Cloud Variables** and click the **ADD VARIABLE** button.

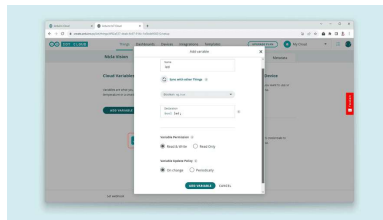
Add one variable with the following characteristics:

Name:

Variable type:

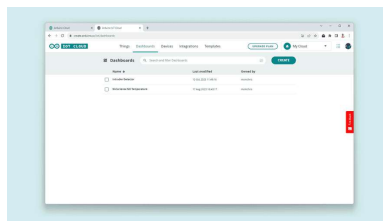
Variable permission:

Variable update policy:



Arduino Cloud "Thing" variable setup

Now, navigate into **Dashboards** and select the **CREATE** button; this will create a new dashboard and give your dashboard a name.



Arduino Cloud Dashboards page

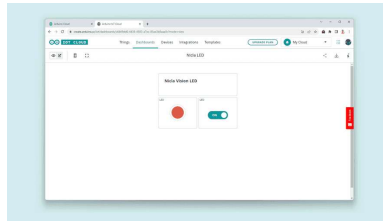
Add the following widgets to your dashboard:

Switch: Name the widget Switch and link it to the variable you created before.

LED: Name the widget LED and link it to the variable you created before.

Sticky Note: Give context to your dashboard with a descriptive title

Your dashboard should look like the following:



Arduino Cloud Dashboard setup

Go back to your **Things** and open the "Thing" you created. In the "Thing" setup page, navigate into **Sketch**, where you should see the online editor.

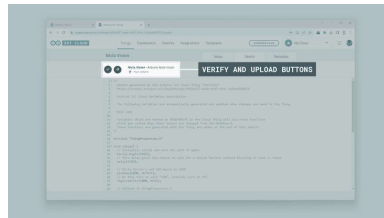
In the generated sketch, define `LEDR` pin as an output in the `setup()` function:

```
1 void setup() {
2   // Initialize serial and wait
3   Serial.begin(9600);
4   // This delay gives the chance
5   delay(1500);
6
7   // Nicla Vision's red LED macro
8   pinMode(LEDR, OUTPUT);
9   // As they turn on with "LOW"
10  digitalWrite(LEDR, HIGH);
11
12  // Defined in thingProperties
13  initProperties();
14
15  // Connect to Arduino Cloud
16  ArduinoCloud.begin(ArduinoIoT
17
18  /*
19   The following function allows
20   related to the state of net
21   the higher number the more
22   The default is 0 (only error
23   Maximum is 4
24  */
25  setDebugMessageLevel(2);
26  ArduinoCloud.printDebugInfo()
27 }
```

In the `onLedChange()` function, which was generated automatically by the Arduino Cloud when the variable `led` was created, you

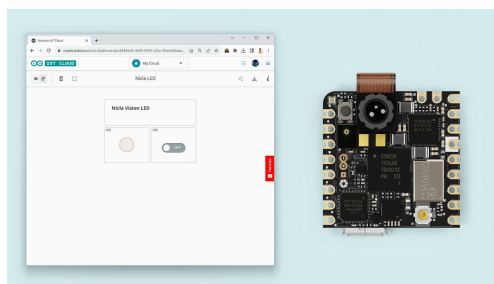

```
1  /*
2   Since Led is READ_WRITE variable, the sketch will be
3   executed every time a new value is received from the serial port.
4  */
5  void onLedChange() {
6    digitalWrite(LED_PIN, !led);
7  }
```

To upload the code to the Nicla Vision from the online editor, click the green **Verify** button to compile the sketch and check for errors, then click the green **Upload** button to program the board with the sketch.



Uploading a sketch to the Nicla Vision in the Arduino Cloud

Navigate into **Dashboards** again, your board should connect to the Wi-Fi® network you defined before (you can follow the connection process with the online editor's integrated Serial Monitor). Your board's red LED (LEDR) should light on or off when the position of the switch changes.



Support

If you encounter any issues or have questions while working with the Nicla Vision, we provide various support resources to help you

Help Center

Explore our [Help Center](#), which offers a comprehensive collection of articles and guides for the Nicla Vision. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Nicla Family Help Center page](#)

Forum

Join our community forum to connect with other Nicla Vision users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Nicla Vision.

[Nicla Vision category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Nicla Vision.

[Contact us page](#)

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can [edit this page here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

